实验二 - 任务分配 (1)

董宜滔
20261014

任务分配

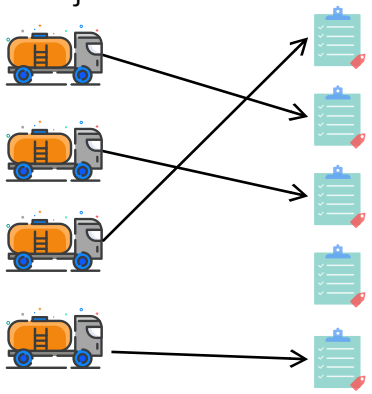
➢ **任务分配定义：**

在给定一组任务和一组工作者的情况下，任务分配是指在满足空间约束、时间约束和 (或) 其他约束的 情况下，为特定目标在任务和工作者之间做出安排的过程。

形式化定义为：给定任务集合 $T = \{t_1, t_2, ..., t_m\}$ 与执行主体集合 $A = \{a_1, a_2, ..., a_n\}$，分配决策可表示为求解优化问题：

$$\min_X \sum_{i=1}^{m}\sum_{j=1}^{n} c_{ij}x_{ij} \quad \text{s.t.} \quad \sum_{j=1}^{n} x_{ij} = 1, \quad \sum_{i=1}^{m} x_{ij} \le r_j, \quad x_{ij} \in \{0,1\}$$

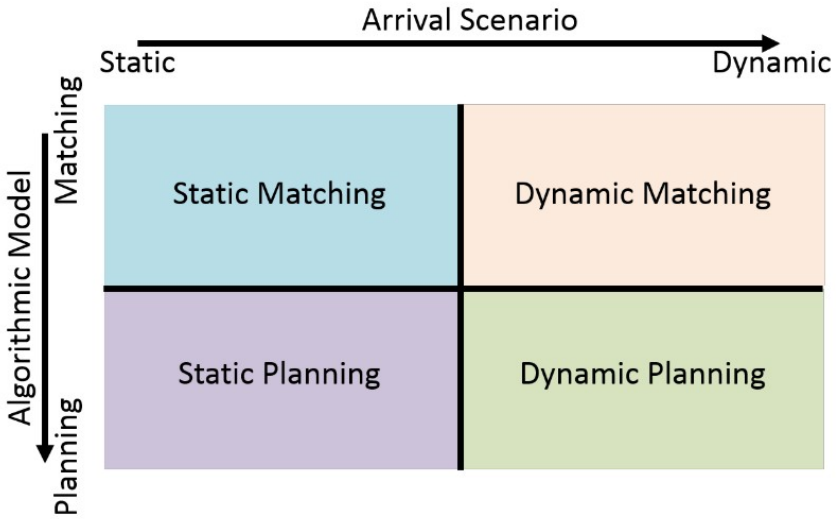
其中 C_{ij} 为任务 t_i 分配给执行主体 a_j 的成本函数， x_{ij} 为二元决策变量 ,r_j 是执行主体 a_j 的容量约束



- 柔性车间调度：通过匹配工序与加工机器，以最大化车间生产量
- 物流运输决策：通过匹配运输订单与运力，以最大化订单匹配数
- 计算任务决策：通过匹配计算任务与运单资源，以最大化资源利用率

任务分配

- 在静态场景中（也称为离线场景），假设平台一开始就知道任务和工人的所有时空信息，包括任务和工人的到达时间和位置
- 在动态场景中（也称为在线场景），任务或工作进程的时空信息只有在它们到达时才知道
- 在匹配模型中，任务分配通常被描述为一个基于二部图的问题。工人和任务可以用二部图中的顶点来表示，工人和任务之间的效用或成本可以用边的权重来表示。
- 在规划模型（也称为调度模型）中，任务分配旨在为每个工人规划一条执行任务的路线，即安排工人执行一系列任务的顺序。



近似比（离线）：

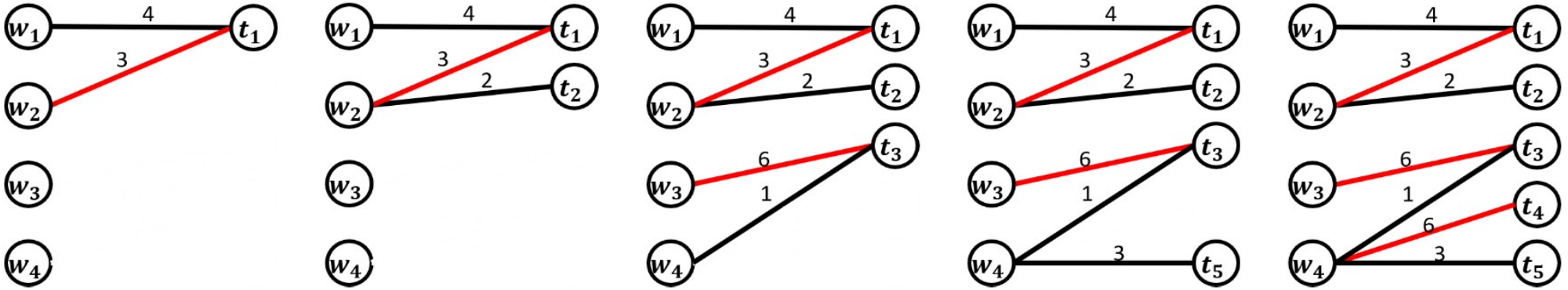
$$\alpha = \frac{\text{算法解的值}}{\text{最优解的值}} \geq 1$$

竞争比（在线）：

$$\beta = \frac{\text{在线算法解的值}}{\text{离线最优解的值}}$$

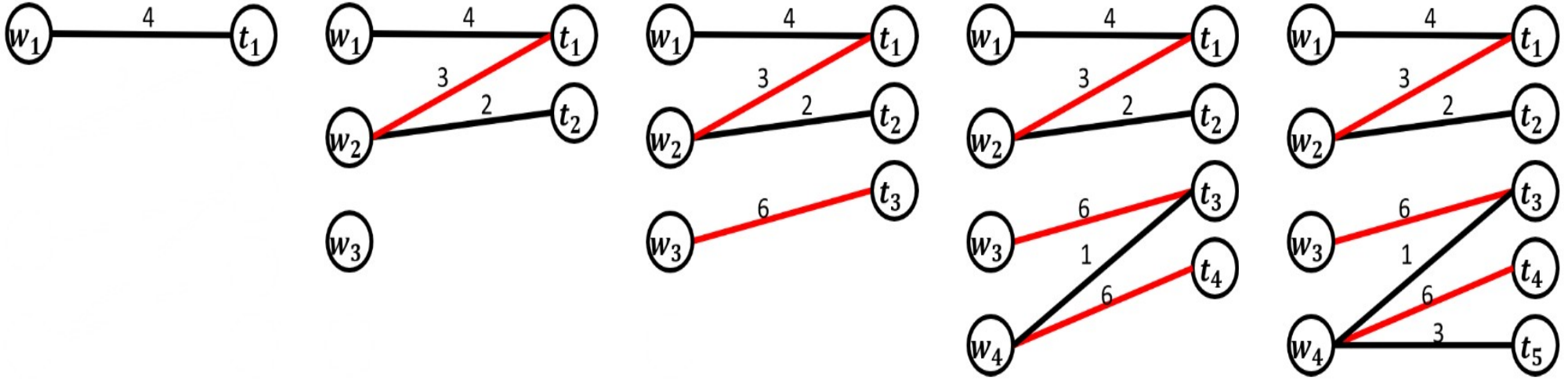
任务分配

单边动态（工人或者订单已知，如邮政派送）



(a) At 7:01. (b) At 7:03. (c) At 7:05. (d) At 7:07. (e) At 7:09.

双边动态（快速匹配问题，如网约车场景）



仿真环境构建

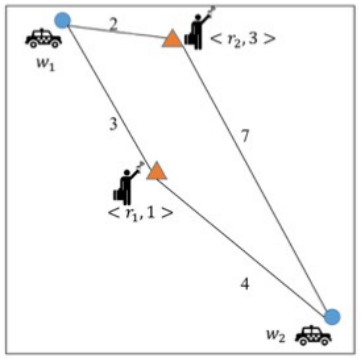
➢ 目标

在一个动态的空间环境中，当任务请求者（如打车乘客）和工作者（如出租车司机）随时间陆续出现时，平台需要为每个请求者分配一个工作者，期望开发一种请求者派单接客时延最小化的算法（公平性）

这是一个双边在线（ two-sided online ）问题，因为工作者和请求者都是动态到达的，平台无法预知未来的情况。这与传统只优化全局指标（如总成本、总效用）的问题有本质区别，它关注的是最差情况下的用户体验

➢ 数据集

我们首先构造一个简单的 2D 欧氏空间（单位：公里）中的任务与工人动态到达场景。



工人和请求者均在 [0, 10] × [0, 10] 的正方形区域内随机出现；工人 / 请求者的移动速度为 1 km/min ，因此两点间的欧氏距离即为行程时间（分钟）；时间戳单位为分钟；总共模拟 T = 20 分钟，每分钟可能有若干请求或工人到达；我们设定总共有 N = 20 个请求和 N = 20 个工人（一一匹配）。

Worker: {id, x, y, t_arrival}

Request: {id, x, y, t_arrival}

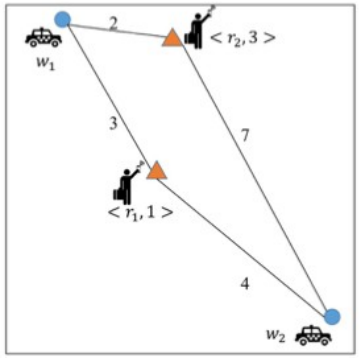
```
def generate_data(N=20, T=20, space=10):
workers_raw, requests_raw = generate_data(N=20, T=20)
```

讨论

- 如果基于到达时间只受距离影响，给定上述仿真器，你将设计什么策略以实现“成本”最小化

- 基于这种策略你将怎么设计算法求解?

贪心策略



- 贪心策略

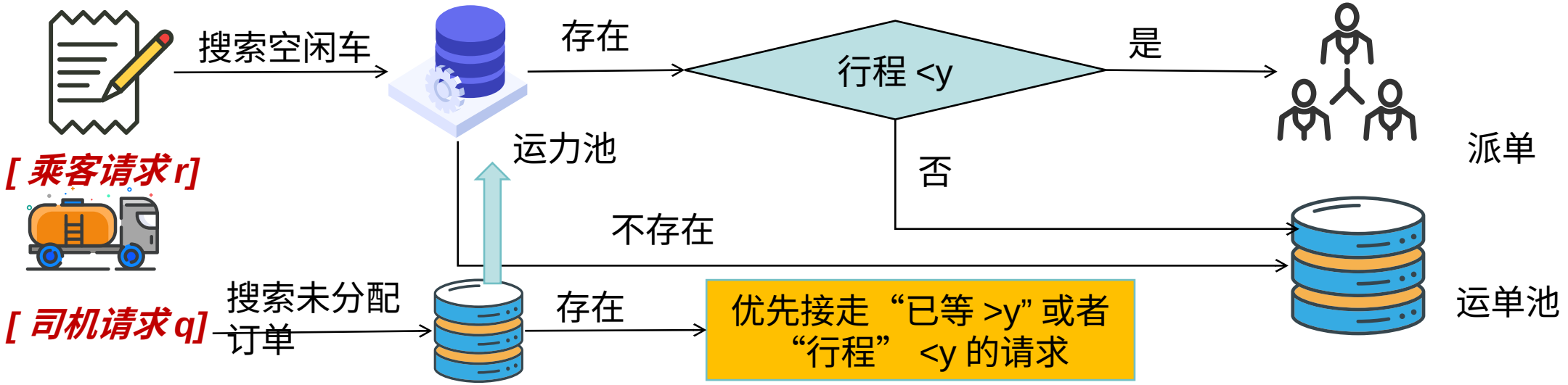
  贪心策略是一种在每一步选择中都做出当前看来最好的选择的算法策略。希望通过一系列局部最优的选择，能够产生全局最优解



拒绝

- 改进贪心策略

  核心思想： “不盲目追求总效率，而是用阈值兜底最差体验”



贪心算法

- 算法输入：
  - 动态到达的司机（含位置、到达时间）
  - 动态到达的货单 / 订单（含位置、发布时间）
  - 延迟阈值 γ （由历史数据或 SLA 设定，如 γ = 15 分钟）
- 核心数据结构：
  - available_workers： 当前空闲司机列表
  - idle_requests： 等待队列（ FIFO ）

```
# 当新订单到达
def new_request(r):
    if 存在空闲司机 and 最近司机行程时间 ≤ γ:
        立即分配该司机
    else:
        将 r 加入等待队列

# 当新司机上线
def new_worker(w):
    for r in 等待队列（按到达顺序）:
        wait = now - r.t
        travel = dist(w, r)
        if wait > γ or (wait + travel) <= γ:
            分配 w 给 r; 跳出循环
    if 未分配:
        w 加入空闲池
```

批次匹配策略

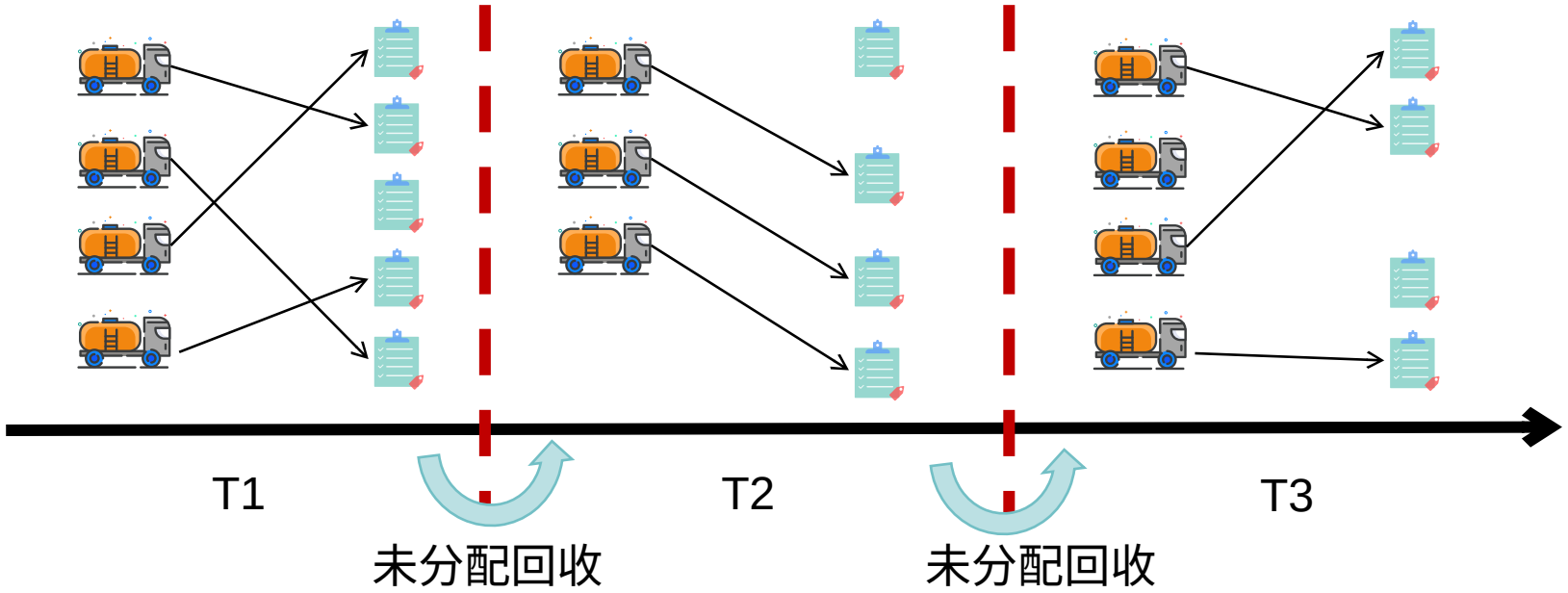
➢ 批次匹配

- 在动态任务分配场景中，传统实时响应方式可能导致局部最优解不佳。
- 批次匹配策略通过引入一个时间窗口 θ，在每个批次结束时进行一次全局优化匹配，从而提升整体资源利用率和用户体验。

➢ 特点

- 批处理机制：每经过固定时间间隔 θ （如 5 分钟），系统会收集所有已到达但尚未分配的任务和工人，并尝试找到当前最佳匹配方案。
- 延迟容忍度：允许一定延迟以换取更优的整体分配结果。
- 离线瓶颈匹配：借鉴离线场景下的瓶颈匹配算法，确保在每次批处理时尽量减少最大延迟。

批次匹配算法



◆ **求解方法：**

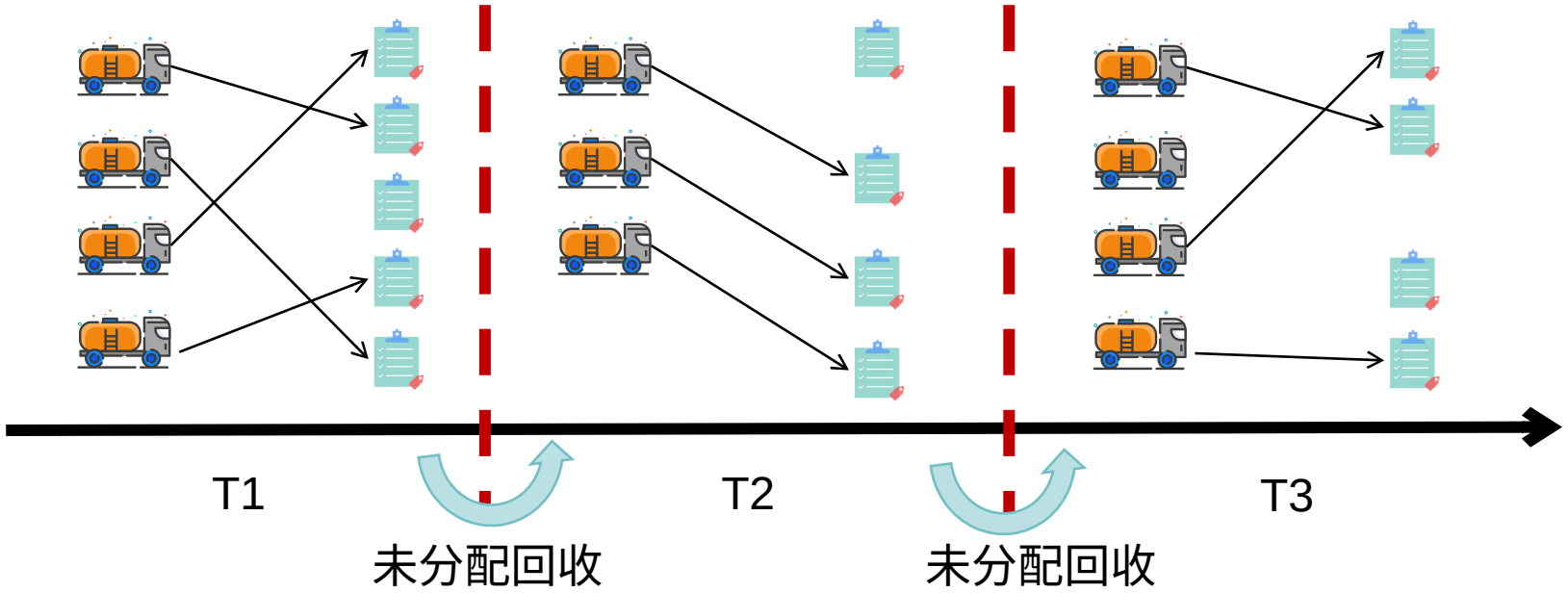
整数规划（松弛项 NP）

进化算法（遗传算法、蚁群算法、灰狼算法 近似解）

组合优化算法（匈牙利算法、 KM 算法 O**3）

贪心算法

批次匹配算法



◆ **求解方法：**

整数规划（松弛项 NP）

进化算法（遗传算法、蚁群算法、灰狼算法 近似解）

组合优化算法（匈牙利算法、 KM 算法 O**3）

贪心算法

讨论

- 有约束了怎么办

作业细则

- **任务分配：**完成 task_assignment.ipynb 的空缺部分与实验报告。
  - 采用贪心策略和批次匹配策略（ KM 算法），构建算法，并对比竞争比
  - 采用 NSGA2 ，探索联合优化最小化接客距离与最大化平台收入（用送客距离表示）的帕累托前沿，并对比竞争比



SCHOOL OF DATA SCIENCE & ENGINEERING
数据科学与工程学院

谢谢！

Email： 52285903012@stu.ecnu.edu.cn